

K-map operators, Boolean forms and cellular automata

Connect four Day 10 questions: operator intent in Kmap4, equivalent SOP/POS forms, and simultaneous next-state logic in Rules 90 and 110.

Kapil Tripathi | Verilog & digital design | 9 pages

CONTENTS / click a row to jump

Entry 161: why Verilog uses instead of +	2-3
Entry 164: SOP, POS and De Morgan's law	4-5
Entry 168: Rule 90 and why nextVal is used	6-7
Entry 172: Rule 110 expression and boundaries	8-9

KEEP THIS IDEA

Use | to combine Boolean alternatives, derive SOP from 1-cells and POS from 0-cells, and keep each automaton's current state separate from its next state.

Tracker entries: 161, 164, 168, 172

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ENTRY 161 / KMAP4

Why the K-map expression uses |, not +

The four bracketed expressions are alternative one-bit conditions. The output must be 1 when any condition is true, so the combining operation is OR.

On paper, Boolean algebra often writes + for OR. In Verilog, the symbols have programming-language meanings: | is bitwise OR, while + is arithmetic addition.

Your accepted construction

[Hint...](#)

Write your solution here


```

1 module top_module(
2     input a,
3     input b,
4     input c,
5     input d,
6     output out
7 );
8 wire w1, w2;
9 xor g1 (w1, c, d) ;
10 xor g2 (w2, a, b);
11 assign out = (~a & ~b) & w1 | (~c & ~d) & w2 | (a & b) & w1 | (c & d) & w2 ; |
12 endmodule
13

```



```

wire w1, w2;
xor g1 (w1, c, d);
xor g2 (w2, a, b);
assign out = (~a & ~b) & w1 | (~c & ~d) & w2
            | ( a & b) & w1 | ( c & d) & w2;

```

Read it aloud as: first condition **OR** second condition **OR** third condition **OR** fourth condition. Parentheses would make the grouping easier to scan, although Verilog already evaluates & before |.

Source: [HDLBits Kmap4](#). The saved submission evidence shows **Status: Success!**.

ENTRY 161 / OPERATOR CHECK

Why + can appear to work here

Arithmetic and OR differ as soon as two true terms overlap. This particular expression hides that difference because its four product terms are mutually exclusive.

Inputs	Bitwise OR	Arithmetic addition
0 and 0	$0 \mid 0 = 0$	$0 + 0 = 0$
0 and 1	$0 \mid 1 = 1$	$0 + 1 = 1$
1 and 1	$1 \mid 1 = 1$	$1 + 1 = \text{binary } 10$

The carry is the warning

```

wire term1 = 1'b1;
wire term2 = 1'b1;
wire      out_or = term1 | term2;           // 1
wire [1:0] sum   = {1'b0,term1} + {1'b0,term2}; // 2'b10

```

If an arithmetic result is kept in only one bit, its carry can be discarded. That can turn $1 + 1$ into a stored 0, while OR must remain 1. The explicit two-bit operands above expose the carry instead of relying on expression-size rules.

Why the submitted terms never overlap

$w1 = c \wedge d$ is 1 only when c, d differ. Its two terms then select either $a, b = 00$ or $a, b = 11$. Similarly, $w2 = a \wedge b$ is 1 only when a, b differ, and its terms require $c, d = 00$ or $c, d = 11$. Those cases cannot be true together, so at most one product term is 1.

What the exhaustive check proves

All 16 input combinations were checked. The submitted expression equals $a \wedge b \wedge c \wedge d$, and never has two active product terms. Therefore + could accidentally match for this exact expression - but | is still the correct, intention-revealing operator.

ENTRY 164 / BOOLEAN FORMS

SOP, POS and De Morgan's law

SOP and POS are two ways to implement the same Boolean function. The difference is the form used to describe it, not a different output.

Form	Structure	K-map route
SOP	OR of product terms; each product term is an AND of literals.	Group output-1 cells, write one product term per group, then OR the terms.
POS	AND of sum terms; each sum term is an OR of literals.	Group output-0 cells, write one sum term per group, then AND the terms.

A precise way to remember them

For SOP, each 1-group creates a condition that makes the function true. For POS, each 0-group creates a sum term that becomes false on that group; ANDing the sum terms blocks the zero cases. Both final expressions still calculate the same output.

Correction to the earlier short note: saying 'SOP describes when the output is 1 and POS describes when it is 0' was too loose. The accurate statement is that the derivation starts from 1-cells for SOP and 0-cells for POS.

De Morgan's law

$$\sim(A \ \& \ B) = (\sim A) \mid (\sim B)$$

$$\sim(A \mid B) = (\sim A) \ \& \ (\sim B)$$

When a NOT moves through a bracket, AND and OR swap, and every literal inside is complemented. This is needed when complementing grouped logic or converting a design into NAND/NOR-friendly form. It is different from factoring or distributing terms.

Why both forms are useful

A truth table may produce fewer gates in one form than the other. The exercise requests both so that you can simplify from the 1-cells and independently from the 0-cells, then recognize that the resulting circuits satisfy the same specified rows.

ENTRY 164 / ACCEPTED SOLUTION

Applying SOP and POS to the problem

The two accepted assignments are equivalent for every input, and they meet all of the problem's specified truth-table rows.

```
assign out_sop = (c & d) | (~a & ~b & c);
assign out_pos = c & (d | ~a) & (d | ~b);
```

Read the names from the outside inward

The SOP has two ANDed product terms joined by OR. The POS has the literal c and two ORed sum terms joined by AND. A single literal may act as a one-literal product or sum term.

Why these two lines are equivalent

```
(c & d) | (~a & ~b & c)
= c & (d | (~a & ~b))      // factor c
= c & (d | ~a) & (d | ~b)  // distribute OR over AND
```

That conversion uses the Boolean distributive identity $X | (Y \& Z) = (X | Y) \& (X | Z)$. It is **not** a De Morgan step because no NOT covers the bracket.

Required output	abcd row numbers	Meaning
1	2, 7, 15	The expression must be true.
0	0, 1, 4, 5, 6, 9, 10, 13, 14	The expression must be false.
Don't care	3, 8, 11, 12	Either value is allowed and may aid simplification.

The four don't-care rows can be chosen as 0 or 1 during minimization. Because of that freedom, more than one equally small POS may be valid; the accepted pair above deliberately implements one consistent value even on the don't-care rows.

Source: HDLBits Exams/ece241_2013_q2.

ENTRY 168 / RULE 90

One whole generation changes at a time

For every cell i , Rule 90 ignores the current center bit and XORs the current left and right neighbours.

$$\text{next_q}[i] = q[i-1] \oplus q[i+1]$$

The eight next-state results, read from neighbourhood 111 down to 000, are 01011010. Interpreted as a binary number, that is decimal 90 - hence the rule's name.

Left	Center	Right	Next center
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	1
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	0

Boundary cells

The imaginary neighbours outside the 512-cell vector are zero: $q[-1] = 0$ and $q[512] = 0$. Therefore $\text{next_q}[0] = q[1]$ and $\text{next_q}[511] = q[510]$.

The same rule as one vector expression

$$\{q[510:0], 1'b0\} \oplus \{1'b0, q[511:1]\}$$

The first shifted vector places $q[i-1]$ at bit i ; the second places $q[i+1]$ at bit i . The inserted zeros implement the two outside boundaries.

All cells must use the same old generation. They are not updated one by one in time. The design computes all 512 next bits as combinational logic, then the clock stores them together.

ENTRY 168 / YOUR ACCEPTED STRUCTURE

Why nextVal is used - and whether it is required

nextVal is a 512-bit combinational candidate for the next generation; q is the 512-bit register holding the current generation.

```
integer i;
reg [511:0] nextVal;

always @(posedge clk) begin
    if (load)
        q <= data;
    else
        q <= nextVal;
end

always @(*) begin
    nextVal[0] = q[1];
    nextVal[511] = q[510];
    for (i = 1; i < 511; i = i + 1)
        nextVal[i] = q[i-1] ^ q[i+1];
end
```

Why you used it

The combinational block calculates every bit of the upcoming state from the unchanged current q. At the rising edge, the clocked block either loads data or captures that complete candidate with the nonblocking assignment `q <= nextVal`. This cleanly separates **current state** from **next-state logic**.

Is nextVal actually required?

No. It is required by this two-process organization, but Rule 90 does not require a separate named signal. The same next-state network can be written directly:

```
always @(posedge clk) begin
    if (load) q <= data;
    else      q <= {q[510:0], 1'b0} ^ {1'b0, q[511:1]};
end
```

Both styles describe essentially the same XOR wiring and 512 state bits. nextVal improves readability and is convenient in a waveform; it does not add another clock cycle or another bank of registers. The synthesis loop is unrolled into hardware rather than executing 510 times over time.

Two important safety points

Every bit of nextVal is assigned (two boundaries plus indices 1 through 510), so the combinational block does not infer a latch. Also avoid updating q in place with blocking assignments inside a loop: later iterations could read already-changed neighbours and would no longer represent a simultaneous generation.

Source: [HDLBits Rule90](#). The current submission panel was rechecked and shows **Status: Success!**.

ENTRY 172 / RULE 110

Rule 110 uses left, center and right

Unlike Rule 90, Rule 110 is asymmetric: swapping the left and right neighbours can change the answer. Keep the vector direction fixed while translating the table.

Left	Center	Right	Next center
1	1	1	0
1	1	0	1
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	1
0	0	0	0

Read from neighbourhood 111 down to 000, the next-state column is 01101110, which is decimal 110.

A compact Boolean form

$$\begin{aligned}\text{next} &= (\sim L \ \& \ C) \mid (C \ \& \ \sim R) \mid (\sim C \ \& \ R) \\ &= (R \wedge C) \mid (R \ \& \ \sim L)\end{aligned}$$

The XOR term handles the rows where center and right differ. The extra $R \ \& \ \sim L$ term adds neighbourhood 011, the remaining row whose output is 1.

Vector direction in your accepted code

$$L = q[i+1] \quad C = q[i] \quad R = q[i-1]$$

With that mapping, $(R \wedge C) \mid (R \ \& \ \sim L)$ becomes exactly the interior expression you submitted. This direction also explains the successful trace from a loaded value of 1: 1, 3, 7, d, 1f, 31, 73, d7, 1fd, 307.

Direction is the important difference from Rule 90. Rule 90 XORs left and right, so mirroring them changes nothing. Rule 110 gives them different roles.

ENTRY 172 / YOUR ACCEPTED STRUCTURE

Why the Rule 110 expression works

The structure again separates the registered current generation `q` from the combinational next generation `nextval`.

```
reg [511:0] nextval;
integer i;

always @(posedge clk) begin
    if (load)
        q <= data;
    else
        q <= nextval;
end

always @(*) begin
    nextval[511] = q[511] | q[510];
    nextval[0]   = q[0];
    for (i = 1; i < 511; i = i + 1)
        nextval[i] = q[i-1] ^ q[i] | ~q[i+1] & q[i-1];
end
```

How Verilog groups the interior expression

```
nextval[i] = (q[i-1] ^ q[i])
             | ((~q[i+1]) & q[i-1]);
```

Unary `~` binds first, then `&`, then `^`, and finally `|`. The added parentheses do not change the hardware; they make the intended $(R \wedge C) \mid (R \wedge \sim L)$ form obvious.

Why the two boundary assignments are different

At bit 0, the outside right neighbour is 0, so the rule reduces to `nextval[0] = q[0]`. At bit 511, the outside left neighbour is 0, so the rule reduces to `nextval[511] = q[511] | q[510]`. These are simplifications of the same rule, not special exceptions.

Why nextval is still useful

As in Rule 90, `nextval` lets all 512 cells be calculated from the unchanged current `q` before one clock edge stores the whole result. Every bit is assigned, so no latch is inferred. The name is convenient, not mandatory; a direct clocked implementation with nonblocking assignments could describe the same state update.

Source: [HDLBits Rule110](#). The current compile-and-simulate panel was rechecked and shows **Status: Success!**.